

령a문s바침제바침장타워바침요수장침제요수상신의장침바침세택취세재석요추상바침요성
 침장바침요상의침제상침세

EP5V구P5r 거결3P5q ~결3~결R5P~거
 Eh공기5pdR0 S결량공5구결893결5?3계5=A8달B=3978D
 Ef l Wf ~구강 관결3 거공각결량경관W공계4 Nf f

Xf l q 0 5f l W Nf f f l Wf ~구강 관결3 거공각결량경관W공계4 Xf l q
 3 5
 f l W %가공각결량경관% 3 5
 /결구강~계관0
 ES그구관공 Xf l q f l W 5
 5

Na9G u개교능 개교

Xf l q /X견공공개가것
 /가견공공개가것

f l W/f ~구강 관결3 거공각결량경관
 /값관

Xf l q

Na9H 교장바침요상의침제

/f ~구강~계관0

% /가계0%
 Xf l q g/결강결0

l 거공각결량경관 E

- 각 노드 u에 대해:
1. u와의 거리에 따라 모든 데이터 점을 정렬
 2. 상위 M개를 candidates로 선택
 3. Spreading-out 휴리스틱 적용:
 - 점진적으로 candidates 중 서로 먼 점들 선택
 - 다양한 방향 커버리지 확보
 - 제한된 수의 간선만 유지

E

Na9N 개요

8E

```
for each point p in dataset:
    // 모든 점과의 거리 계산
    distances = calculate_distances(p, all_points)

    // 상위 K개 근접 점 선택 (K >> M)
    candidates = top_K_nearest(distances)

    // p의 이웃으로 설정
    neighbors[p] = candidates
```

9E이 거리와 각 점의 근접도

```
function spreading_out(candidates, M):
    result = []
    selected = [first_element(candidates)]

    while |result| < M:
        // 현재까지 선택된 점들로부터 가장 먼 점 선택
        farthest = argmax_distance_to_selected(candidates - selected)
        selected.add(farthest)
        result.append(farthest)

    return result
```

: E / k 개 근접도 계산

```
for each point p with neighbors N(p):
    for each neighbor q in N(p):
        // q에서 p로 역방향 링크 추가
        add_reverse_link(q, p)
```

=E / 0

```
for iteration in 1 to num_iterations:
    for each point p:
        // 현재 이웃들
        current_neighbors = neighbors[p]

        // 이웃의 이웃들 탐색 (2-hop 친구)
        friend_candidates = union(neighbors[neighbor]
                                   for neighbor in current_neighbors)

        // Spreading-out으로 새로운 이웃 선택
        new_neighbors = spreading_out(friend_candidates, M)

        // 더 나으면 업데이트
        if quality(new_neighbors) > quality(current_neighbors):
            neighbors[p] = new_neighbors
```

Na9P

E

```
procedure search_NSg(query_q, start_node, ef):
    visited = {start_node}
    candidates = {start_node}
    w = {start_node} // 결과 집합

    while not empty(candidates):
        // 현재 가장 가까운 점
        current = get_nearest_from_candidates(candidates)

        // 더 이상의 개선 불가능하면 종료
        if distance(current, q) > distance(farthest_in_w, q):
```

```
break

// current의 이웃 탐색
for neighbor in neighbors[current]:
    if neighbor not in visited:
        visited.add(neighbor)
        d = distance(neighbor, q)

        // 더 좋은 점을 찾았으면 추가
        if d < distance(farthest_in_w, q) or |w| < ef:
            candidates.add(neighbor)
            w.add(neighbor)

        // 결과 집합 유지
        if |w| > ef:
            w.remove(farthest_point)

return top_K_nearest_from_w(w, K)
```

Naïve 3-요제 통계의 문제

$$\frac{\sum_{f \in W} \sum_{X \in f} \sum_{q \in X} \text{distance}(q, f)}{|W|}$$

entry_point = centroid of all points
또는
entry_point = random point from dataset

Naïve W

$$\frac{\sum_{e \in E} \sum_{X \in f} \sum_{q \in X} \text{distance}(q, e)}{|E|} \quad \text{vs} \quad \frac{\sum_{c \in C} \sum_{X \in f} \sum_{q \in X} \text{distance}(q, c)}{|C|}$$

간접 거리를 통한
간접 거리를 통한

$$\frac{\sum_{e \in E} \sum_{X \in f} \sum_{q \in X} \text{distance}(q, e)}{|E|} \quad \text{vs} \quad \frac{\sum_{c \in C} \sum_{X \in f} \sum_{q \in X} \text{distance}(q, c)}{|C|}$$

Naïve

초기 그래프: $O(n * \log n)$ 거리 계산 + 정렬
정렬: $O(\log n)$ per point
최적화: $O(n * M * K)$ 반복 개선
전체: $O(n * K * \log n) \sim O(n * K * \text{상수})$

HNSW: 대략 64 바이트/점 (M=16)
NSG: 대략 32 바이트/점 (M=16)
→ NSG가 약 50% 메모리 절약

E

평균: $O(\log n)$ 또는 $O(\sqrt{n})$
ef 파라미터에 따라 선형 계수 변동

Na9개교추첨개교능

	Xf l q	f l W
가가가가가가가가가가가		
	나	
6		

Na9개교

E

Xf l q ?7,

E

E

Xf l q

E

/ 0

Na9GF개교

E

6

Xf l q

E

E

Na9GG개교

E

f l W

E

E

E

85f l W	E f l W	3	
X f l q	L		
95	E	L	
	L		
: 5	E f l W		
L			
=5	E	f l W	L
	L		
?5	f l W E	/	0 f l W
A5	E X f l q	f l W	L
B5	E	f l W	L
C5	E f l W X f l q		L
D5	E f l W	3	L